GUÍA DEL ALUMNO

Una app, muchas páginas. Una sola carga.

TaskFlow con páginas: **rutas tipadas, rutas anidadas, guardias, lazy loading y View Transitions.**

Configuración y uso de Vue Router

Vue 3.5 · Vue Router 5 · Pinia 4

Oscar Alvarado

Frontend Tech Lead

Universidad San Martín de Porres

Lima, Perú

ANTES DE EMPEZAR · QUÉ VAS A ENCONTRAR EN ESTA GUÍA

TaskFlow, ahora con páginas

Seguimos con **TaskFlow**, la app de las cuatro sesiones del módulo. La sesión pasada aprendió a guardar su **estado** (Pinia); hoy le damos **rutas** con Vue Router. Cada tema es una página corta de teoría y, enseguida, el código para escribirlo.

Los cinco bloques de hoy

R1
Vue Router y rutas tipadas

¿Cómo le damos varias páginas a TaskFlow?createRouter · RouterView

R2
Rutas anidadas y parámetros

¿Cómo mostramos el detalle de cualquier proyecto?children · :id

R3
Guardias de navegación

¿Cómo dejamos pasar solo a quien corresponde?beforeEach · meta

R4
Lazy loading y prefetch

¿Cómo cargamos solo lo que se necesita?import() · chunks

R5
Transiciones de vista

¿Cómo animamos el cambio de página?View Transitions API

TaskFlow a lo largo del módulo

SESIÓN	QUÉ SE AGREGA A TASKFLOW
1 · Estado	Stores de tareas, tema y sesión, datos de un servidor simulado y Tailwind.
2 · Vue Router (hoy)	Páginas: proyectos, detalle con pestañas, login, guardias, carga diferida y transiciones.
3 · Firebase	El servidor simulado se reemplaza por Firebase; los stores y las rutas no cambian.
4 · Acceso	Permisos reales por rol (admin / miembro), también en el servidor.

Cómo seguir la clase

- Parte de `src/checkpoints/inicio/` del proyecto de ejemplos: es TaskFlow al terminar la sesión 1.
- Cada bloque termina en un **checkpoint** (`r1` … `r5`): una carpeta `src/` completa. Si te pierdes, pégala dentro de tu `src/` y sigue.
- Usa **Chrome o Edge** en el último bloque (View Transitions). Node 22.18 o superior, VS Code con **Vue (Official)**.

De una pantalla a varias páginas

CONCEPTO createRouter

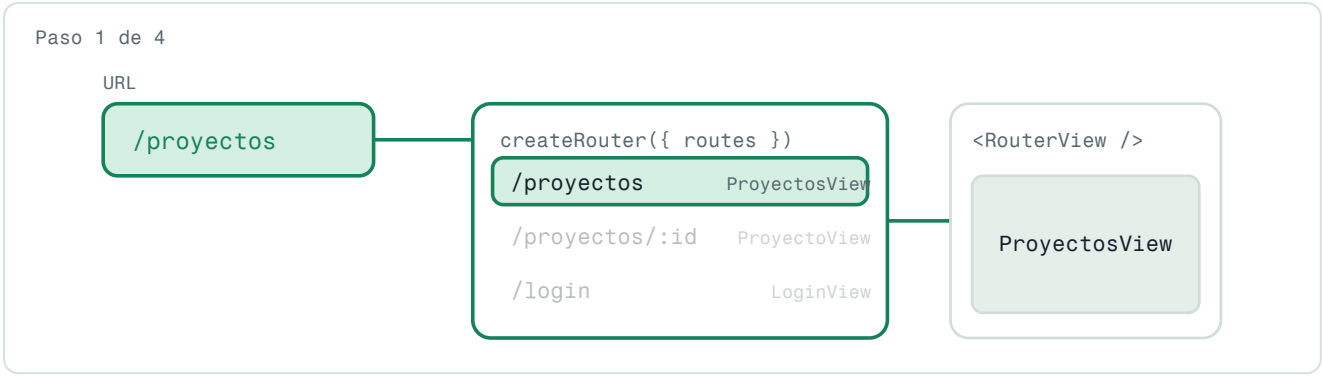
TaskFlow tiene una sola pantalla. ¿Cómo le damos varias páginas sin recargar el navegador?

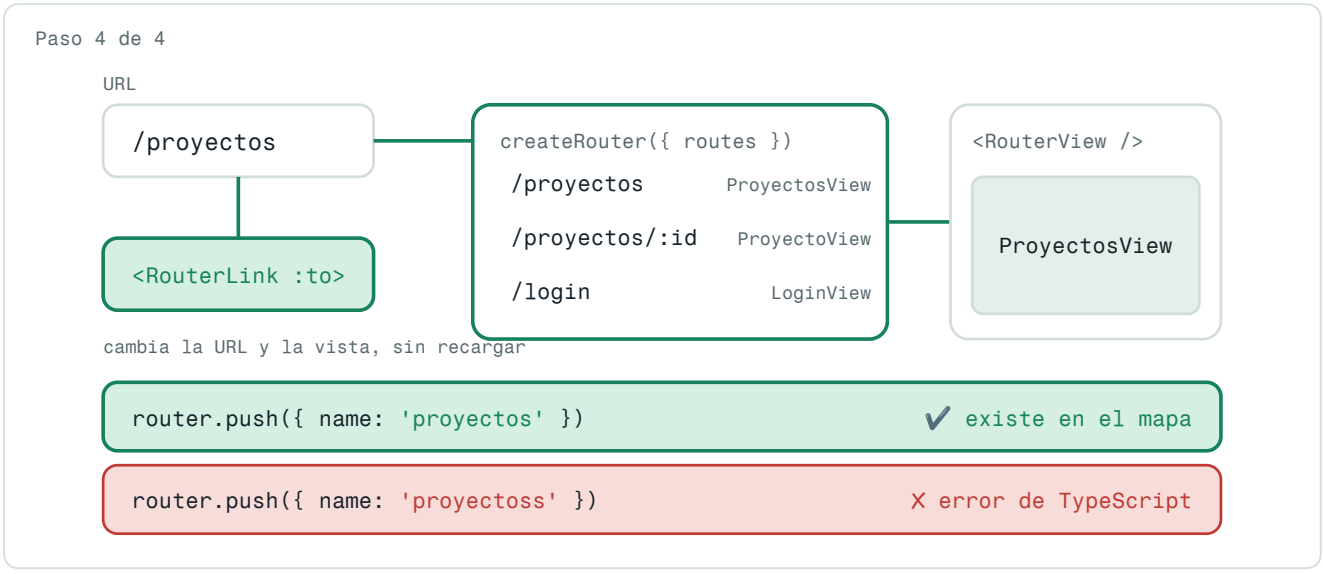
QUÉ ES

El enrutador oficial de Vue: asocia cada **URL** con un componente y lo dibuja en `<RouterView>`, sin recargar la página.

PARA QUÉ SE USA

Tener páginas, enlaces y el botón «atrás» en una app de una sola página. Con un **mapa de rutas tipado**, el editor valida nombres y parámetros mientras escribes.





Cómo funciona, paso a paso

1. **Una URL, una vista.** El usuario pide `/proyectos`; el router busca esa ruta y dibuja su componente en `<RouterView>`.
2. **El mapa de rutas.** `createRouter({ history, routes })`: cada ruta es `{ path, name, component }`. `createWebHistory()` da URLs limpias, sin `#`.
3. **Navegar sin recargar.** `<RouterLink :to>` cambia la URL y la vista; la app, sus stores y su estado siguen vivos.
4. **Rutas tipadas.** Un `RouteRecordInfo` por ruta le enseña a TypeScript los nombres y parámetros: `push({ name: 'proyectoss' })` falla al escribir, no en producción.

REGLA

Navega **por nombre** (`{ name: 'proyectos' }`), no por texto de URL: si la URL cambia, nada se rompe, y TypeScript avisa si el nombre no existe.

LO QUE AGREGAMOS A TASKFLOW

- instalar `vue-router` y crear el router
- rutas con nombre y su mapa tipado
- `RouterView` y `RouterLink` en TaskFlow

Bloque R1 de 5 · Router y rutas tipadas

QUÉ ESCRIBES Y QUÉ PEGAS

✍️ **Escribes:** router/tipos.ts, router/index.ts y main.ts. 📋 **Pegas:** App.vue, BarraSuperior y las vistas (carpeta piezas/r1).

1. **Terminal** — `npm install vue-router`.
2. **router/tipos.ts** — el **mapa de rutas tipado**: un `RouteRecordInfo` por ruta y el `declare module` que se lo entrega a Vue Router.
3. **router/index.ts y main.ts** — `createRouter` con `createWebHistory()` y las rutas; la 404 va *al final*. `app.use(router)` lo registra.
4. **App.vue y views/** — ⚡ `App.vue` solo deja el marco y un `<RouterView />`; la pantalla de tareas pasa a `TareasView.vue`.
5. **BarraSuperior.vue** — `<RouterLink :to="{ name }">`. Prueba un nombre mal escrito: el editor lo subraya.

src/router/tipos.ts

```ts
import type { RouteRecordInfo } from 'vue-router'

/**
 * Mapa de rutas de TaskFlow: nombre → ruta y parámetros.
 * Con esto el editor valida `router.push({ name: … })` y `<RouterLink :to>` mientras
   escribes.
 */
export interface RutasTaskFlow {
  inicio: RouteRecordInfo<'inicio', '/'>
  proyectos: RouteRecordInfo<'proyectos', '/proyectos'>
  tareas: RouteRecordInfo<'tareas', '/tareas'>
  'no-encontrada': RouteRecordInfo<
    'no-encontrada',
    '/:pathMatch(.*)*',
    { pathMatch: string | string[] },
    { pathMatch: string[] }
  >
}

declare module 'vue-router' {
  interface TypesConfig {
    RouteNamedMap: RutasTaskFlow
  }
}
```

src/router/index.ts

```ts
import { createRouter, createWebHistory } from 'vue-router'
import NoEncontrada from '../views/NoEncontrada.vue'
import ProyectosView from '../views/ProyectosView.vue'
import TareasView from '../views/TareasView.vue'

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', name: 'inicio', redirect: { name: 'proyectos' } },
    { path: '/proyectos', name: 'proyectos', component: ProyectosView },
    { path: '/tareas', name: 'tareas', component: TareasView },
    // Siempre al final: captura cualquier URL que no coincidió con las anteriores
    { path: '/:pathMatch(.*)*', name: 'no-encontrada', component: NoEncontrada },
  ],
})
```

src/main.ts

```ts
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
import { router } from './router'
import './assets/main.css'

const pinia = createPinia()

createApp(App).use(pinia).use(router).mount('#app')
```

src/App.vue

```vue
<script setup lang="ts">
import { useProyectosStore } from './stores/proyectos'
import { useTareasStore } from './stores/tareas'
import BarraSuperior from './components/BarraSuperior.vue'

// Los datos se piden una vez, aquí; las vistas solo los muestran
useTareasStore().cargar()
useProyectosStore().cargar()
</script>

<template>
  <div class="min-h-screen bg-slate-50 text-slate-800 dark:bg-slate-900 dark:text-
   slate-100">
    <BarraSuperior />

    <main class="mx-auto flex max-w-2xl flex-col gap-4 p-6">
      <RouterView />
    </main>
  </div>
</template>
```

BLOQUE R2 · RUTAS ANIDADAS Y PARÁMETROS

El detalle de cualquier proyecto

CONCEPTO `children · :id`

Ya hay páginas. ¿Cómo mostramos el detalle de CUALQUIER proyecto sin una ruta por cada uno?

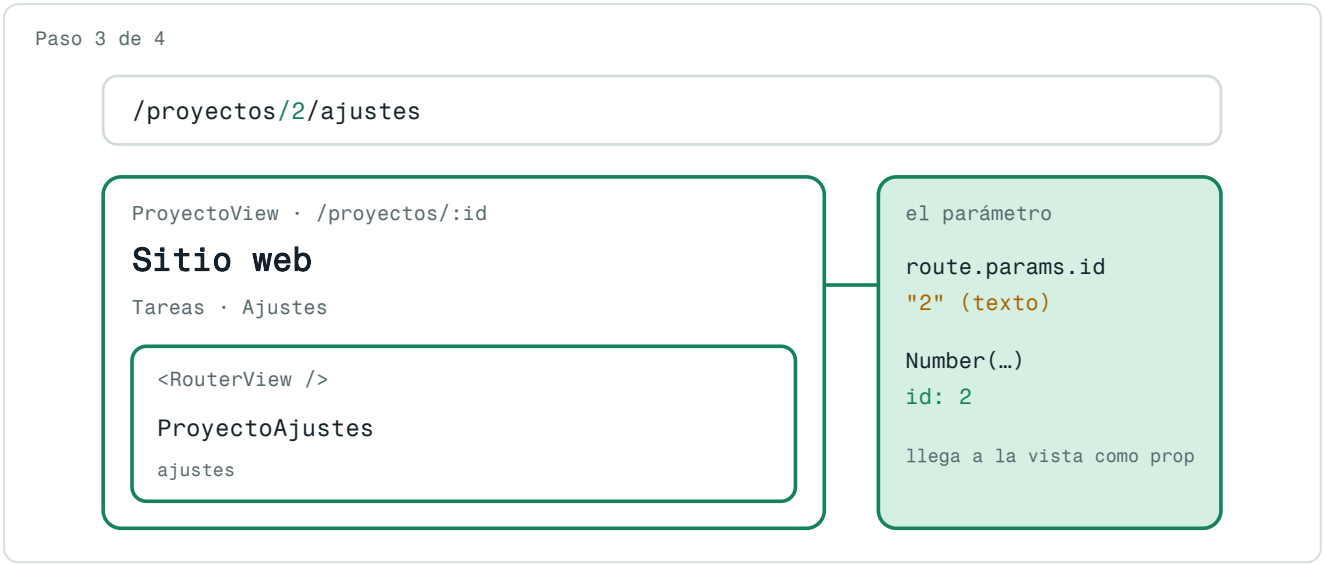
QUÉ ES

Una ruta **padre** que contiene rutas **hijas** (`children`) y segmentos dinámicos como `:id` que cambian en cada URL.

PARA QUÉ SE USA

Reutilizar un marco (encabezado, pestañas) y cambiar solo lo de adentro: `/proyectos/2` y `/proyectos/2/ajustes` comparten el encabezado del proyecto.





Cómo funciona, paso a paso

1. **Una URL, varias piezas.** `:id` es el segmento que cambia (`2`) y `ajustes` es el hijo.
2. **El padre pone el marco.** `ProyectoView` dibuja el encabezado y deja un `<RouterView>` donde entra el hijo.

3. **El parámetro, como prop.** `route.params.id` es `"2"`; la ruta lo convierte con `props: (route) => …` y la vista recibe `id: 2`.
4. **Cambiar de pestaña.** Al ir de Tareas a Ajustes, el padre se queda y solo se reemplaza el hijo.

REGLA

Los parámetros llegan siempre como **texto**: conviértelos una vez (`Number(route.params.id)`) y pásalos como **props**, para que la vista no dependa del router.

LO QUE AGREGAMOS A TASKFLOW

- `/proyectos/:id` con hijos `tareas` y `ajustes`
- parámetro → prop numérico
- pestañas con `RouterLink` y ruta 404

Bloque R2 de 5 · Rutas anidadas y parámetros

QUÉ ESCRIBES Y QUÉ PEGAS

✍️ **Escribes:** las rutas children y props, y los parámetros en el mapa tipado. 📋 **Pegas:** ProyectoView, ProyectoAjustes, TareasView y los demás (piezas/r2).

1. **views/ProyectoView.vue** — la **ruta padre**: encabezado, pestañas y un `<RouterView />` para el hijo. Recibe `id` como número.
2. **router/tipos.ts y router/index.ts** — `children`: el hijo con `path: ''` es la pestaña por defecto. `props: (route) => …` convierte `route.params.id` en número *una sola vez*.
3. **TareasView, NuevaTarea, ProyectosView** — reciben `proyectoId` y filtran. El enlace pasa `params: { id }`: el mapa tipado **exige** el parámetro.
4. **Navegador** — entra a un proyecto, cambia de pestaña (el encabezado no se redibuja) y prueba `/proyectos/99`.

src/views/ProyectoView.vue

```
<script setup lang="ts">
import { computed } from 'vue'
import { useProyectosStore } from '../stores/proyectos'

const props = defineProps<{ id: number }>()

const proyectos = useProyectosStore()
const proyecto = computed(() => proyectos.porId(props.id))
</script>

<template>
  <RouterLink :to="{ name: 'proyectos' }" class="text-sm text-sky-600 dark:text-sky-400">
    ← Proyectos
  </RouterLink>
```

```vue
  <p v-if="proyectos.cargando" class="animate-pulse text-center text-slate-500">
    Cargando proyecto…
  </p>
  <p v-else-if="!proyecto" class="text-center text-slate-500">Este proyecto no existe.</p>
  <template v-else>
    <h2 class="text-2xl font-bold">{{ proyecto.nombre }}</h2>

    <nav class="flex gap-2 border-b border-slate-200 text-sm dark:border-slate-700">
      <RouterLink
        :to="{ name: 'proyecto-tareas', params: { id } }"
        class="-mb-px border-b-2 border-transparent px-3 py-2 hover:text-sky-600"
        active-class="border-sky-600 font-semibold text-sky-600"
      >
        Tareas
      </RouterLink>
      <RouterLink
        :to="{ name: 'proyecto-ajustes', params: { id } }"
        class="-mb-px border-b-2 border-transparent px-3 py-2 hover:text-sky-600"
        active-class="border-sky-600 font-semibold text-sky-600"
      >
        Ajustes
      </RouterLink>
    </nav>

    <!-- Aquí se dibuja la ruta hija: tareas o ajustes -->
    <RouterView />
  </template>
</template>
```

src/router/index.ts

```ts
import { createRouter, createWebHistory, type RouteLocationNormalized } from 'vue-router'
import NoEncontrada from '../views/NoEncontrada.vue'
import ProyectoAjustes from '../views/ProyectoAjustes.vue'
import ProyectosView from '../views/ProyectosView.vue'
import ProyectoView from '../views/ProyectoView.vue'
import TareasView from '../views/TareasView.vue'

// Los parámetros de la URL siempre llegan como texto: se convierten aquí, una sola vez
const idDe = (route: RouteLocationNormalized) => Number((route.params as { id: string }).id)

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', name: 'inicio', redirect: { name: 'proyectos' } },
    { path: '/proyectos', name: 'proyectos', component: ProyectosView },
    {
      // Ruta padre: pinta el encabezado del proyecto y, dentro, un <RouterView> para el
   hijo
      path: '/proyectos/:id',
      component: ProyectoView,
      props: (route) => ({ id: idDe(route) }),
      children: [
        {
          path: '',
          name: 'proyecto-tareas',
          component: TareasView,
          props: (route) => ({ proyectoId: idDe(route) }),
        },
        {
          path: 'ajustes',
          name: 'proyecto-ajustes',
```

```
          component: ProyectoAjustes,
        },
      ],
    },
    // Siempre al final: captura cualquier URL que no coincidió con las anteriores
    { path: '/:pathMatch(.*)*', name: 'no-encontrada', component: NoEncontrada },
  ],
})
```

src/views/TareasView.vue

```
<script setup lang="ts">
import { computed } from 'vue'
import { storeToRefs } from 'pinia'
import { red } from '../services/tareasApi'
import { useSesionStore } from '../stores/sesion'
import { useTareasStore } from '../stores/tareas'
import TareaCard from '../components/TareaCard.vue'
import NuevaTarea from '../components/NuevaTarea.vue'
import FiltroTareas from '../components/FiltroTareas.vue'

const tareas = useTareasStore()
const { visibles, cargando, error } = storeToRefs(tareas)
const sesion = useSesionStore()

const props = defineProps<{ proyectoId: number }>()
const deEsteProyecto = computed(() =>
  visibles.value.filter((t) => t.proyectoId === props.proyectoId),
)
</script>

<template>
  <label class="flex items-center gap-2 text-sm text-slate-500">
    <input v-model="red.falla" type="checkbox" />
    Simular error de red
  </label>

  <NuevaTarea :proyecto-id="proyectoId" />
  <FiltroTareas />

  <p v-if="cargando" class="animate-pulse text-center text-slate-500">Cargando tareas…</p>
  <div
    v-else-if="error"
    class="flex items-center justify-between gap-3 rounded border border-red-300 bg-red-50
   px-4 py-3 text-red-800 dark:border-red-800 dark:bg-red-950 dark:text-red-200"
  >
    <span>{{ error }}</span>
    <button
      type="button"
      class="rounded bg-red-600 px-3 py-1 text-sm text-white hover:bg-red-700"
      @click="tareas.cargar()"
    >
      Reintentar
    </button>
  </div>
  <template v-else>
    <ul class="flex flex-col gap-3">
      <TareaCard
        v-for="t in deEsteProyecto"
        :key="t.id"
        :tarea="t"
        :puede-eliminar="sesion.esAdmin"
```

```
        @avanzar="tareas.avanzar"
        @eliminar="tareas.eliminar"
      />
    </ul>
    <p v-if="deEsteProyecto.length === 0" class="text-center text-slate-500">
      No hay tareas en esta vista.
    </p>
  </template>
</template>
```

Quién puede entrar adónde

CONCEPTO beforeEach

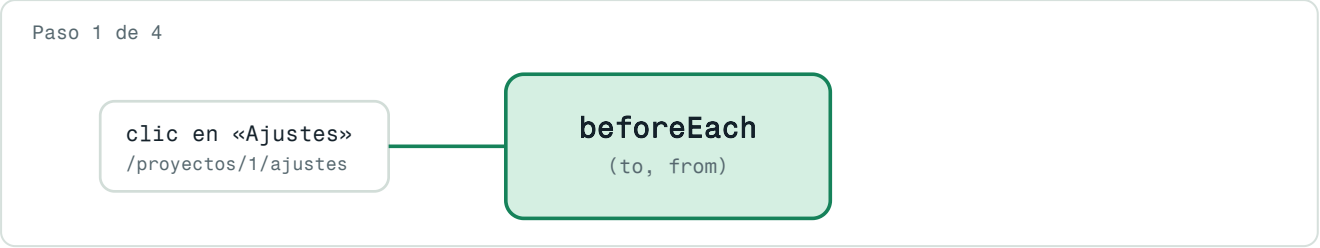
Cualquiera puede escribir /proyectos en la barra. ¿Cómo dejamos pasar solo a quien corresponde?

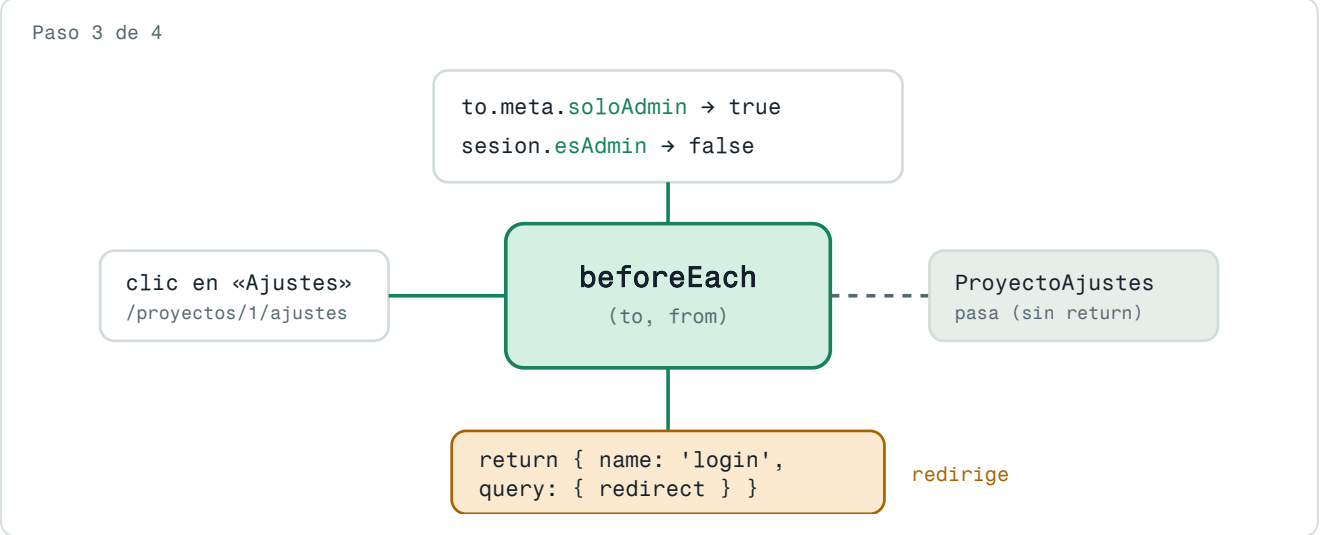
QUÉ ES

Una función que el router ejecuta **antes de cada navegación**. Recibe a dónde vas (`to`) y decide: dejar pasar, redirigir o cancelar.

PARA QUÉ SE USA

Proteger páginas según la sesión o el rol, marcándolas con `meta` (`requiereAuth` , `soloAdmin`) en vez de repetir el `if` en cada vista.





Cómo funciona, paso a paso

1. **Antes de entrar.** Cada clic o `push` pasa por `router.beforeEach((to, from) => …)` antes de cambiar la vista.
2. **Lee el contexto.** `to.meta` dice qué exige la ruta; el store de sesión dice quién eres. Se llama *dentro* de la guardia, no al inicio del archivo.
3. **Devuelve la decisión.** Nada = pasa · una ruta = redirige (con `query.redirect` para volver después) · `false` = cancela.
4. **Y no basta.** Un usuario puede editar el JavaScript o llamar al servidor directo: la guardia solo cuida la experiencia.

REGLA

Una guardia es **experiencia de usuario, no seguridad**: el código del navegador se puede saltar. Quien decide de verdad es el servidor (sesión 4).

LO QUE AGREGAMOS A TASKFLOW

- `meta` tipado en las rutas
- `beforeEach` con redirect a `/login`
- volver a donde querías ir al entrar

Bloque R3 de 5 · Guardias de navegación

QUÉ ESCRIBES Y QUÉ PEGAS

✍️ **Escribes:** RouteMeta, la ruta de login y beforeEach. 📋 **Pegas:** LoginView y BarraSuperior (piezas/r3).

1. **router/tipos.ts** — extiende `RouteMeta` con `requiereAuth` y `soloAdmin`: `to.meta` queda tipado.
2. **views/LoginView.vue** — ⚡ lee `route.query.redirect` para volver adonde ibas; solo acepta rutas internas.
3. **router/index.ts** — `meta` en las rutas y `router.beforeEach`. **`useSesionStore()` va dentro de la guardia**, no arriba del archivo.
4. **BarraSuperior.vue** — ⚡ «Entrar» es un enlace al login y `salir()` navega a mano.

OJO

Una guardia es **experiencia de usuario, no seguridad**: quien edite el JavaScript la salta. El servidor tiene que volver a validar (sesión 4).

src/router/index.ts

```
import { createRouter, createWebHistory, type RouteLocationNormalized } from 'vue-router'
import { useSesionStore } from '../stores/sesion'
import LoginView from '../views/LoginView.vue'
import NoEncontrada from '../views/NoEncontrada.vue'
import ProyectoAjustes from '../views/ProyectoAjustes.vue'
import ProyectosView from '../views/ProyectosView.vue'
import ProyectoView from '../views/ProyectoView.vue'
import TareasView from '../views/TareasView.vue'

// Los parámetros de la URL siempre llegan como texto: se convierten aquí, una sola vez
const idDe = (route: RouteLocationNormalized) => Number((route.params as { id: string }).id)

export const router = createRouter({
  history: createWebHistory(),
  routes: [
```

```
    { path: '/', name: 'inicio', redirect: { name: 'proyectos' } },
    {
      path: '/proyectos',
      name: 'proyectos',
      component: ProyectosView,
      meta: { requiereAuth: true },
    },
    { path: '/login', name: 'login', component: LoginView },
    {
      // Ruta padre: pinta el encabezado del proyecto y, dentro, un <RouterView> para el
   hijo
      path: '/proyectos/:id',
      component: ProyectoView,
      meta: { requiereAuth: true },
      props: (route) => ({ id: idDe(route) }),
      children: [
        {
          path: '',
          name: 'proyecto-tareas',
          component: TareasView,
          props: (route) => ({ proyectoId: idDe(route) }),
        },
        {
          path: 'ajustes',
          name: 'proyecto-ajustes',
          component: ProyectoAjustes,
          meta: { soloAdmin: true },
        },
      ],
    },
    // Siempre al final: captura cualquier URL que no coincidió con las anteriores
    { path: '/:pathMatch(.*)*', name: 'no-encontrada', component: NoEncontrada },
  ],
})

// Guardia global: corre antes de CADA navegación. Devolver una ruta la redirige; no
   devolver nada la deja pasar.
router.beforeEach((to) => {
  const sesion = useSesionStore()

  if (to.meta.requiereAuth && !sesion.autenticado) {
    // Guardamos a dónde quería ir para volver después de entrar
    return { name: 'login', query: { redirect: to.fullPath } }
  }
  if (to.meta.soloAdmin && !sesion.esAdmin) {
    return { name: 'proyectos' }
  }
  if (to.name === 'login' && sesion.autenticado) {
    return { name: 'proyectos' }
  }
})
```

src/views/LoginView.vue

```
<script setup lang="ts">
import { useRoute, useRouter } from 'vue-router'
import { USUARIOS_DEMO, useSesionStore } from '../stores/sesion'

const sesion = useSesionStore()
const route = useRoute()
const router = useRouter()
```

```
// `redirect` viene de la guardia. Solo aceptamos rutas internas ("/algo"), nunca "//otro-
   sitio"
const redirect = route.query.redirect
const destino =
  typeof redirect === 'string' && redirect.startsWith('/') && !redirect.startsWith('//')
    ? redirect
    : '/proyectos'

function entrar(id: number) {
  sesion.entrar(id)
  router.push(destino)
}
</script>

<template>
  <section class="mx-auto flex w-full max-w-sm flex-col gap-4 py-8">
    <h2 class="text-2xl font-bold">Entrar a TaskFlow</h2>
    <p v-if="route.query.redirect" class="text-sm text-slate-500">
      Para ver <code>{{ route.query.redirect }}</code> primero inicia sesión.
    </p>
    <button
      v-for="u in USUARIOS_DEMO"
      :key="u.id"
      type="button"
      class="rounded-lg border border-slate-200 bg-white px-4 py-3 text-left shadow-sm
   hover:shadow-md dark:border-slate-700 dark:bg-slate-800"
      @click="entrar(u.id)"
    >
      <span class="font-medium">Entrar como {{ u.nombre }}</span>
      <span class="ml-2 text-sm text-slate-500">{{ u.rol }}</span>
    </button>
  </section>
</template>
```

Descargar solo lo que se usa

CONCEPTO `import()`

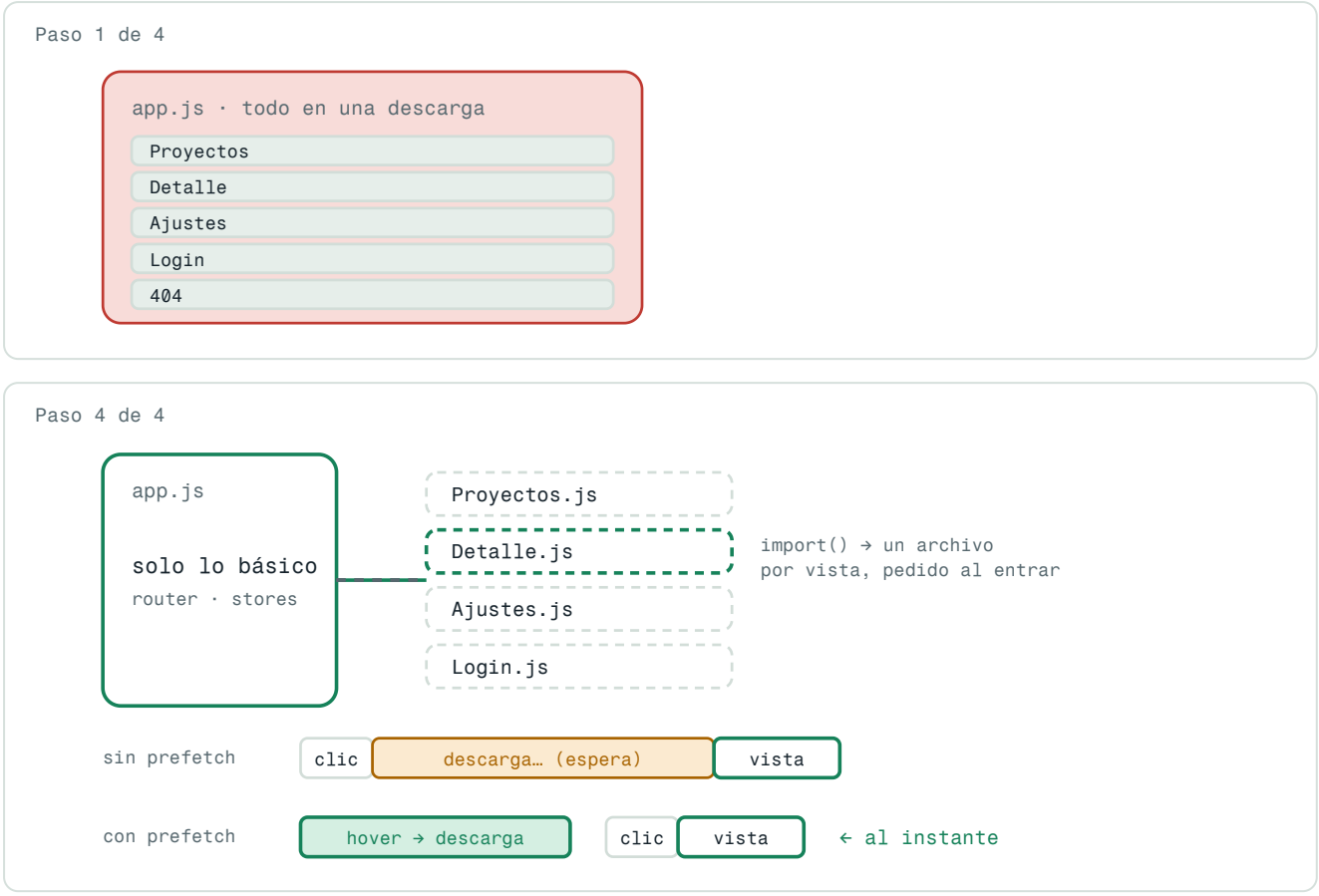
Cada vista nueva engorda el JavaScript inicial. ¿Cómo cargamos solo lo que el usuario necesita?

QUÉ ES

Un **import dinámico** en la ruta: `component: () => import('./X.vue')`. El bundler separa la vista en su propio archivo (*chunk*) y se descarga al entrar.

PARA QUÉ SE USA

Que la primera carga sea pequeña (**lazy loading**) y que, con **prefetch**, el chunk ya esté listo antes del clic: al pasar el cursor o cuando el navegador está en reposo.



Cómo funciona, paso a paso

1. **Sin lazy.** Todas las vistas viajan en un solo archivo: el usuario descarga páginas que quizá nunca abra.
2. **Con `import()`.** El archivo principal queda pequeño y cada vista es un chunk aparte que se pide cuando hace falta.
3. **El costo.** Al navegar por primera vez hay que descargar el chunk: se nota una espera.
4. **Prefetch.** Si lo pedimos antes (`mouseenter` o `requestIdleCallback`), el chunk ya está en caché al hacer clic.

REGLA

Lazy en **vistas** (rutas), no en componentes pequeños. Y prefetch con medida: precarga lo probable, no todo.

LO QUE AGREGAMOS A TASKFLOW

- vistas con `import()`
- ver los chunks en `vite build` y en Network
- prefetch al pasar el cursor y en reposo

Bloque R4 de 5 · Lazy loading y prefetch

QUÉ ESCRIBES Y QUÉ PEGAS

✍️ **Escribes:** los import() de las rutas. 📋 **Pegas:** router/precarga.ts, ProyectosView y main.ts (piezas/r4).

1. **router/index.ts** — cambia cada vista por `component: () => import('…')`.
2. **Terminal** — `npm run build`: un archivo por vista y un principal pequeño. En `Network` cada página pide su archivo al entrar.
3. **router/precarga.ts, ProyectosView.vue, main.ts** — ⚡ `precargar()` llama a la función `import()` de la ruta antes del clic: al pasar el cursor (`mouseenter`) y cuando el navegador está en reposo.

src/router/index.ts

```
import { createRouter, createWebHistory, type RouteLocationNormalized } from 'vue-router'
import { useSesionStore } from '../stores/sesion'

// Los parámetros de la URL siempre llegan como texto: se convierten aquí, una sola vez
const idDe = (route: RouteLocationNormalized) => Number((route.params as { id: string }).id)

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', name: 'inicio', redirect: { name: 'proyectos' } },
    {
      path: '/proyectos',
      name: 'proyectos',
      component: () => import('../views/ProyectosView.vue'),
      meta: { requiereAuth: true },
    },
    { path: '/login', name: 'login', component: () => import('../views/LoginView.vue') },
    {
      // Ruta padre: pinta el encabezado del proyecto y, dentro, un <RouterView> para el
   hijo
      path: '/proyectos/:id',
      component: () => import('../views/ProyectoView.vue'),
      meta: { requiereAuth: true },
      props: (route) => ({ id: idDe(route) }),
      children: [
```

```
        {
          path: '',
          name: 'proyecto-tareas',
          component: () => import('../views/TareasView.vue'),
          props: (route) => ({ proyectoId: idDe(route) }),
        },
        {
          path: 'ajustes',
          name: 'proyecto-ajustes',
          component: () => import('../views/ProyectoAjustes.vue'),
          meta: { soloAdmin: true },
        },
      ],
    },
    // Siempre al final: captura cualquier URL que no coincidió con las anteriores
    {
      path: '/:pathMatch(.*)*',
      name: 'no-encontrada',
      component: () => import('../views/NoEncontrada.vue'),
    },
  ],
})

// Guardia global: corre antes de CADA navegación. Devolver una ruta la redirige; no
   devolver nada la deja pasar.
router.beforeEach((to) => {
  const sesion = useSesionStore()

  if (to.meta.requiereAuth && !sesion.autenticado) {
    // Guardamos a dónde quería ir para volver después de entrar
    return { name: 'login', query: { redirect: to.fullPath } }
  }
  if (to.meta.soloAdmin && !sesion.esAdmin) {
    return { name: 'proyectos' }
  }
  if (to.name === 'login' && sesion.autenticado) {
    return { name: 'proyectos' }
  }
})
```

src/router/precarga.ts

```ts
import type { RouteLocationRaw } from 'vue-router'
import { router } from '.'

/**
 * Prefetch: baja el chunk de la vista ANTES de que el usuario navegue.
 * Cada ruta con `component: () => import(…)` guarda esa función; llamarla ya pide el
   archivo.
 * Si el chunk ya llegó, el navegador lo tiene en caché y no repite la descarga.
 */
export function precargar(to: RouteLocationRaw) {
  for (const registro of router.resolve(to).matched) {
    const cargador = registro.components?.default
    if (typeof cargador === 'function') (cargador as () => Promise<unknown>)()
  }
}

/** Ejecuta `tarea` cuando el navegador está libre (Safari no tiene
   requestIdleCallback) */
export function enReposo(tarea: () => void) {
  if ('requestIdleCallback' in window) requestIdleCallback(tarea)
  else setTimeout(tarea, 1500)
}
```

Que la navegación se sienta fluida

CONCEPTO View Transitions

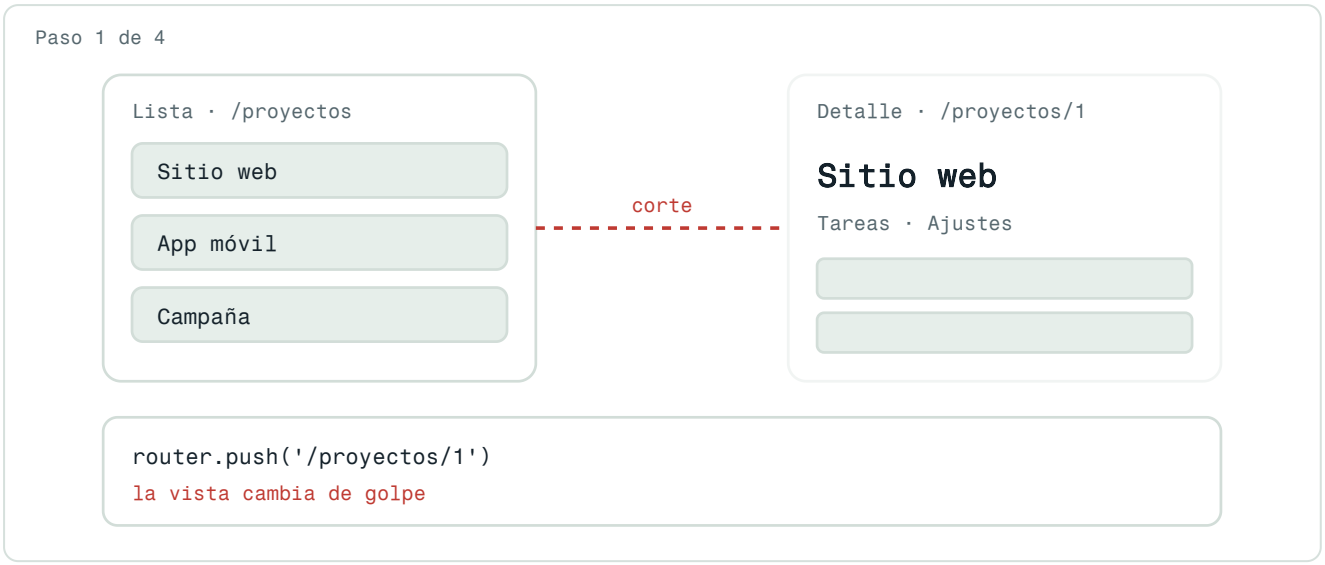
Las páginas ya cargan rápido, pero cambian de golpe. ¿Cómo animamos la navegación sin librerías?

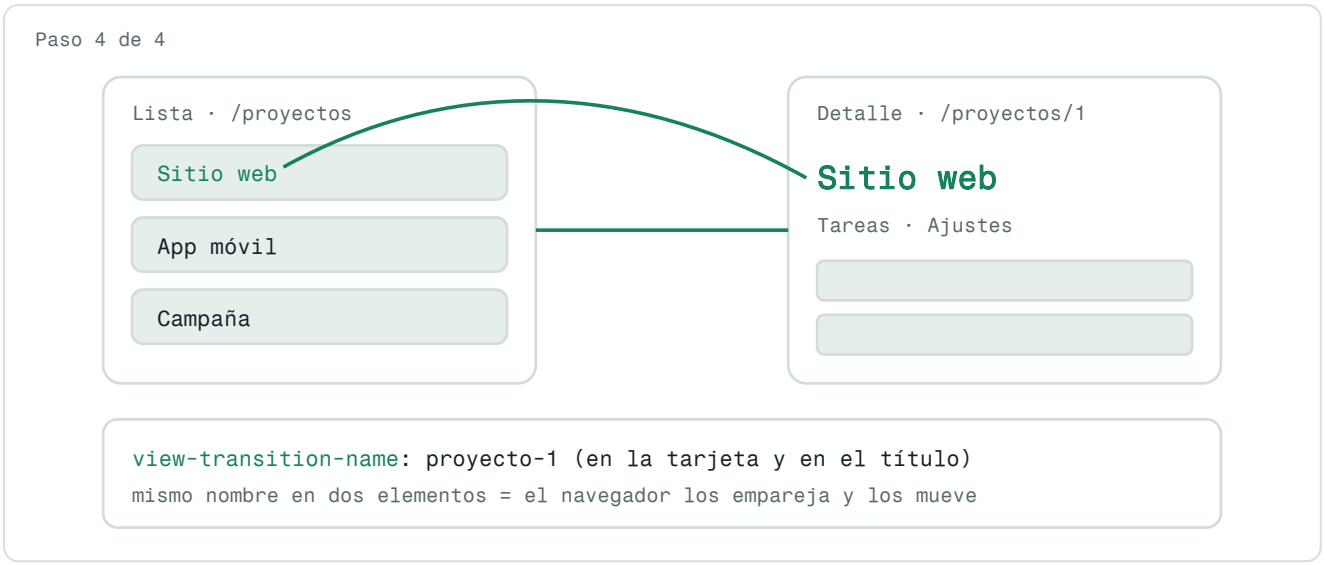
QUÉ ES

Una API del navegador: `document.startViewTransition()` toma una «foto» de la vista, ejecuta el cambio y anima de la foto vieja a la nueva.

PARA QUÉ SE USA

Transiciones entre páginas hechas con CSS: un fundido gratis y, con `view-transition-name`, un elemento que «viaja» de la lista al detalle.





Cómo funciona, paso a paso

1. **Sin transición.** El router cambia la vista y el usuario ve un corte.
2. **La foto y el cambio.** `startViewTransition(callback)` congela una captura de la vista actual y *después* corre el callback, que cambia la vista. El navegador anima de una foto a la otra

(`::view-transition-old` y `::view-transition-new`).

3. **El router lo coordina.** En `beforeResolve` la navegación espera a que exista la foto; al terminar (`afterEach`) se libera la animación. Si el router cambiara la vista antes, la foto vieja ya sería la nueva y no se vería nada.
4. **Un elemento que viaja.** El mismo `view-transition-name` en la tarjeta y en el título del detalle los empareja: el título se desplaza de uno a otro.

REGLA

La foto vieja se toma **antes** de cambiar la ruta. Es **mejora progresiva**: sin soporte, la navegación funciona igual. Respeta `prefers-reduced-motion`.

LO QUE AGREGAMOS A TASKFLOW

- `router/transiciones.ts`: una transición en cada navegación
- `view-transition-name` en la lista y el detalle
- apagarla con «reducir movimiento»

Bloque R5 de 5 · Transiciones de vista

QUÉ ESCRIBES Y QUÉ PEGAS

✍️ **Escribes:** instalarTransiciones(router) y el CSS. 📋 **Pegas:** router/transiciones.ts y las vistas (piezas/r5).

1. **router/transiciones.ts** — `router.beforeResolve` espera a que el navegador tome la foto (`document.startViewTransition`) y solo entonces deja cambiar la vista; `router.afterEach` la libera cuando Vue ya pintó. Se instala con `instalarTransiciones(router)`.
2. **ProyectosView.vue y ProyectoView.vue** — `viewTransitionName: 'proyecto-' + id` en el título de la tarjeta *y* en el `<h2>` del detalle: el título viaja. El nombre es único por pantalla.
3. **assets/main.css** — ajusta la duración con `::view-transition-old(root)` y apágala con `prefers-reduced-motion`.

OJO

La foto «vieja» tiene que tomarse **antes** de cambiar la ruta. Por eso no usamos `<RouterLink view-transition>`: con vistas ya cargadas solo anima la primera vez.

src/router/transiciones.ts

```ts
import { nextTick } from 'vue'
import { START_LOCATION, type Router } from 'vue-router'

/**
 * View Transitions en TODA navegación (también el botón «atrás»).
 *
 * El orden importa: el navegador toma la foto «vieja» cuando se llama a
   startViewTransition. Si el
 * router ya cambió la vista antes de ese momento, la foto vieja es igual a la nueva y
   no se ve nada.
 * Por eso la navegación espera en `beforeResolve` hasta que el navegador tomó la foto,
   y solo
 * entonces cambia la vista; la transición termina cuando Vue ya pintó la vista nueva.
 */
export function instalarTransiciones(router: Router) {
  let terminar: (() => void) | undefined

  router.beforeResolve((to, from) => {
    const sinAnimacion =
      from === START_LOCATION ||
      !('startViewTransition' in document) ||
      matchMedia('(prefers-reduced-motion: reduce)').matches
    if (sinAnimacion) return

    return new Promise<void>((foto) => {
      document.startViewTransition(() => {
        foto() // ya hay foto vieja: la navegación puede cambiar la vista
        return new Promise<void>((resolver) => (terminar = resolver))
      })
    })
  })

  // Corre al final de cada navegación, también si falló o se canceló
  router.afterEach(() => {
    nextTick(() => {
      terminar?.()
      terminar = undefined
    })
  })
}
```

src/views/ProyectosView.vue

```vue
<script setup lang="ts">
import { precargar } from '../router/precarga'
import { useProyectosStore } from '../stores/proyectos'
import { useTareasStore } from '../stores/tareas'

const proyectos = useProyectosStore()
const tareas = useTareasStore()

const cuantas = (proyectoId: number) =>
  tareas.tareas.filter((t) => t.proyectoId === proyectoId && t.estado !== 'hecha').length
</script>

<template>
```

```
  <h2 class="text-2xl font-bold">Proyectos</h2>

  <p v-if="proyectos.cargando" class="animate-pulse text-center text-slate-500">
    Cargando proyectos…
  </p>
  <ul v-else class="flex flex-col gap-3">
    <li v-for="p in proyectos.proyectos" :key="p.id">
      <RouterLink
        :to="{ name: 'proyecto-tareas', params: { id: p.id } }"
        @mouseenter="precargar({ name: 'proyecto-tareas', params: { id: p.id } })"
        @focus="precargar({ name: 'proyecto-tareas', params: { id: p.id } })"
        class="flex items-baseline justify-between gap-3 rounded-lg border border-slate-200
   bg-white p-4 shadow-sm transition hover:shadow-md dark:border-slate-700 dark:bg-slate-
   800"
      >
        <!-- Mismo nombre en la lista y en el detalle: el navegador anima de uno a otro -->
        <span class="text-lg font-medium" :style="{ viewTransitionName: `proyecto-${p.id}`
   }">
          {{ p.nombre }}
        </span>
        <span class="text-sm text-slate-500">{{ cuantas(p.id) }} por hacer</span>
      </RouterLink>
    </li>
  </ul>
</template>
```

src/assets/main.css

```
@import 'tailwindcss';

/* El tema oscuro se activa con la clase "dark" en <html>, no con el sistema operativo
   */
@custom-variant dark (&:where(.dark, .dark *));

/* View Transitions: la raíz se funde en 0.25 s y los elementos con nombre viajan de
   una vista a otra */
::view-transition-old(root),
::view-transition-new(root) {
  animation-duration: 0.25s;
}

/* Quien pide reducir el movimiento no ve animación */
@media (prefers-reduced-motion: reduce) {
  ::view-transition-group(*),
  ::view-transition-old(*),
  ::view-transition-new(*) {
    animation: none !important;
  }
}
```

CIERRE · PARA SEGUIR

Siete preguntas, siete respuestas

PREGUNTA	RESPUESTA
¿Cómo le damos páginas a la app?	`createRouter` · `RouterView` · `RouterLink`
¿Cómo evitamos nombres de ruta mal escritos?	mapa de rutas tipado (`RouteRecordInfo`)
¿Cómo se muestra el detalle de cualquier proyecto?	ruta padre con `children` y `:id`
¿Cómo llega el parámetro a la vista?	texto en `route.params` → prop numérica
¿Cómo protegemos páginas?	`beforeEach` + `meta` (UX, no seguridad)
¿Cómo cargamos solo lo necesario?	`() => import()` + prefetch
¿Cómo animamos la navegación?	`beforeResolve` + `startViewTransition` + `view-transition-name`

Para practicar

Agrega la pestaña `miembros` en `/proyectos/:id/miembros`: una vista cargada con `import()`, visible para cualquier usuario con sesión, y añádela al mapa tipado. Bonus: haz que el enlace a Ajustes solo se muestre al admin.

Glosario

TÉRMINO	EN UNA FRASE
Router	Asocia cada URL con una vista y cambia de página sin recargar.
`RouterView` · `RouterLink`	Donde se dibuja la vista de la ruta; el enlace que navega sin recargar.
Ruta con nombre	Una ruta a la que se navega por su `name`, no por su texto de URL.
Ruta anidada	Ruta hija que se dibuja dentro del `RouterView` de su padre.
Parámetro de ruta	Segmento dinámico (`:id`) que llega en `route.params` como texto.
Guardia · `meta`	Función que corre antes de navegar; datos propios de cada ruta que ella consulta.
Chunk · lazy loading	Archivo de JavaScript de una vista; cargarlo solo al entrar.
Prefetch	Pedir un chunk antes de que el usuario lo necesite.
View Transition	API del navegador que anima entre dos estados de la página con CSS.

Para repasar

Vue Router · documentación — `router.vuejs.org`

Rutas anidadas	router.vuejs.org
Guardias de navegación	router.vuejs.org
Carga diferida de rutas	router.vuejs.org
Rutas tipadas	router.vuejs.org
View Transitions API	developer.mozilla.org